AI & MACHINE LEARNING TRAINING

Assignment — Week 4, Day 5

Multiple Linear Regression & Feature Selection

Date	30-Oct-2026	**Level**	Intermediate
Duration	5 Hours	**Deliverable**	Improved regression model

1. Learning Objectives

- Explain multiple linear regression and how it differs from simple linear regression.
- Train a model using several input features.
- Select and compare useful feature combinations.
- Calculate and interpret residuals and inspect a residual plot.
- Evaluate models using MAE, MSE, RMSE and R^2.
- Identify common errors, improve the model and explain decisions in your own words.

2. Learn the Concepts

A. Simple vs Multiple Linear Regression

Simple linear regression uses one input feature, such as house area, to predict price. Multiple linear regression uses two or more features together, such as area, bedrooms, age and distance from the city centre.

$$\textbf{Predicted price} = b_0 + b_1(\text{area}) + b_2(\text{bedrooms}) + b_3(\text{age}) + b_4(\text{distance})$$

b_0 is the intercept. Each b value is a learned coefficient. A coefficient estimates the change in predicted price for a one-unit change in that feature, while the other included features are held constant. It does not prove cause and effect.

B. Feature Selection

Feature selection means choosing which input columns the model should use. More features do not always mean a better model. Irrelevant, duplicate or misleading features can make a model harder to understand and may reduce performance on new data.

- Choose features that make practical sense for the problem.
- Compare a baseline model with models using different feature combinations.
- Use the same train/test split for every experiment.
- Do not repeatedly use test-set results to select features; reserve the test set for final evaluation.
- Never include the target column (price) among the input features.

C. Residuals

A residual is the difference between the actual value and the predicted value.

$$\textbf{Residual} = \textbf{Actual value} - \textbf{Predicted value}$$

A positive residual means the model predicted too low. A negative residual means it predicted too high. Residuals close to zero are useful, but you must also inspect whether they form a pattern.

- Random scatter around zero is generally a healthy sign for a linear model.
- A curve may mean a straight-line model is missing a non-linear relationship.
- A funnel shape may mean error spread changes across the prediction range.
- Investigate large residuals; do not automatically delete them.

3. Five-Hour Work Plan

Time	Activity
Hour 1	Re-run yesterday's house-price notebook and record baseline metrics.
Hour 2	Prepare the data and train a model using all four candidate features.
Hour 3	Run feature-selection experiments and compare three feature combinations.
Hour 4	Calculate residuals, create a residual plot and investigate unusual errors.
Hour 5	Choose the best justified model, test predictions and complete reflection.

4. Practical Task — Improve the House-Price Model

Continue from yesterday's house-price assignment. Reuse your dataset if it has the columns below. Otherwise, create house_prices.csv using this fictional learning dataset (not real market prices).

area_sqft	bedrooms	age_years	distance_km	price_inr
800	2	10	8	4200000
950	2	8	6	5100000
1100	2	5	5	6300000
1250	3	7	7	6900000
1400	3	4	4	8500000
1550	3	12	9	7900000
1700	3	6	5	9800000
1850	3	3	3	11500000
2000	4	8	6	11200000
2150	4	5	4	12900000
2300	4	2	3	14800000
2450	4	10	8	13500000
2600	4	4	5	15800000
2750	4	7	6	16000000
2900	5	3	2	18500000
3050	5	9	7	17400000
3200	5	4	4	19800000
3350	5	6	5	20500000
3500	5	2	3	22500000
3650	5	8	6	21800000

Step 1 — Load and inspect

Load the CSV with pandas. Check head(), shape, info(), missing values and describe(). Set price_inr as the target; the other four columns are candidate features.

Code:

```
import pandas as pd
```

```
data = {
   "area_sqft": [800, 950, 1100, 1250, 1400, 1550, 1700, 1850, 2000, 2150,
                 2300, 2450, 2600, 2750, 2900, 3050, 3200, 3350, 3500, 3650],
   "bedrooms": [2, 2, 2, 3, 3, 3, 3, 3, 4, 4,
                4, 4, 4, 4, 5, 5, 5, 5, 5, 5],
   "age_years": [10, 8, 5, 7, 4, 12, 6, 3, 8, 5,
                 2, 10, 4, 7, 3, 9, 4, 6, 2, 8],
   "distance_km": [8, 6, 5, 7, 4, 9, 5, 3, 6, 4,
                   3, 8, 5, 6, 2, 7, 4, 5, 3, 6],
   "price_inr": [4200000, 5100000, 6300000, 6900000, 8500000,
                 7900000, 9800000, 11500000, 11200000, 12900000,
                 14800000, 13500000, 15800000, 16000000, 18500000,
                 17400000, 19800000, 20500000, 22500000, 21800000]
}
df = pd.DataFrame(data)
df.to_csv("house_prices.csv", index=False)
print("CSV file created successfully!")
df = pd.read_csv("house_prices.csv")
print(df.head())
print(df.shape)
df.info()
print(df.isnull().sum())
print(df.describe())
# Step 2 — Create one train/test split
# Use train_test_split(test_size=0.2, random_state=42). Split before fitting. Reuse this exact split for all models to make the comparison fair.
from sklearn.model_selection import train_test_split
X = df[["area_sqft", "bedrooms", "age_years", "distance_km"]]
y = df["price_inr"]
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("\nTraining target shape:", y_train.shape)
```

```
print("Testing target shape:", y_test.shape)

# Step 3 — Train three models

# •	Model A — Baseline: area_sqft only.

# •	Model B — Multiple regression: area_sqft, bedrooms, age_years and distance_km.

# •	Model C — Your selected feature combination. Choose fewer or different features and justify your decision.

# For each model, train LinearRegression with fit(), predict the test data, then calculate MAE, MSE, RMSE and R².

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import numpy as np

def evaluate_model(features, model_name):

    # Select training and testing features from the same split

    X_train_selected = X_train[features]

    X_test_selected = X_test[features]

    # Create model

    model = LinearRegression()

    # Train model

    model.fit(X_train_selected, y_train)

    # Make predictions

    y_pred = model.predict(X_test_selected)

    # Calculate evaluation metrics

    mae = mean_absolute_error(y_test, y_pred)

    mse = mean_squared_error(y_test, y_pred)

    rmse = np.sqrt(mse)

    r2 = r2_score(y_test, y_pred)

    print("\n", model_name)

    print("Features:", features)

    print("MAE:", mae)

    print("MSE:", mse)

    print("RMSE:", rmse)

    print("R2 Score:", r2)

    return model, y_pred, {

            "Model": model_name,

            "Features": features,
```

```
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2
    }
model_A, pred_A, result_A = evaluate_model(
    ["area_sqft"], "Model A"
)
model_B, pred_B, result_B = evaluate_model(
    ["area_sqft", "bedrooms", "age_years", "distance_km"],
    "Model B"
)
model_C, pred_C, result_C = evaluate_model(
    ["area_sqft", "age_years", "distance_km"],
    "Model C"
)
# Step 4 — Compare the models
# Create a table containing model name, selected features, MAE, RMSE and R². Explain which model is best for this dataset. Do not assume that the model with the most features must perform best.
results = pd.DataFrame([
    result_A,
    result_B,
    result_C
])
print(results.to_string(index=False))print(results)
```

Explanation: Model A achieved the lowest MAE of ₹418,258.94. Model C achieved the lowest RMSE of ₹543,913.07 and the highest R^2 of 0.9944. I selected Model C because it performed better in RMSE and R^2 while using only three features. However, Model A had a lower average absolute prediction error.

```
# Step 5 — Calculate and plot residuals
# For your chosen model, calculate residual = actual price − predicted price. Display at least 10 rows with actual price, predicted price and residual. Plot predicted price on the x-axis and residual on the y-axis, with a horizontal reference line at zero.
# Select Model C
best_model = model_C
best_features = ["area_sqft", "age_years", "distance_km"]
# Predict prices for training data
```

```
train_predictions = best_model.predict(
    X_train[best_features]
)
# Calculate training residuals
train_residuals = y_train - train_predictions
# Create residual DataFrame
residual_df = pd.DataFrame({
    "Actual_Price": y_train,
    "Predicted_Price": train_predictions,
    "Residual": train_residuals
})
# Display at least 10 observations
print(residual_df.head(10))
# Plot residuals
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 5))
plt.scatter(
    train_predictions,
    train_residuals
)
plt.axhline(y=0, color="red", linestyle="--")
plt.xlabel("Predicted House Price")
plt.ylabel("Residuals")
plt.title("Training Residual Plot")
plt.grid(True)
plt.show()
# Step 6 — Investigate errors
# Find the three largest absolute residuals. Inspect their feature values and suggest a possible reason for each error. Do not remove observations without a valid data-based reason.
# Calculate absolute residuals
residual_df["Absolute_Residual"] = (
    residual_df["Residual"].abs()
)
# Sort from largest to smallest
largest_errors = residual_df.sort_values(
```

```
    by="Absolute_Residual",
    ascending=False
)
# Add original feature values
error_analysis = largest_errors.join(
    X_train
)
# Display three largest errors
print(error_analysis.head(3))
```

output:

```
CSV file created successfully!
   area_sqft  bedrooms  age_years  distance_km  price_inr
0       800        2         10            8    4200000
1       950        2          8            6    5100000
2      1100        2          5            5    6300000
3      1250        3          7            7    6900000
4      1400        3          4            4    8500000
(20, 5)
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   area_sqft    20 non-null     int64
 1   bedrooms     20 non-null     int64
 2   age_years    20 non-null     int64
 3   distance_km  20 non-null     int64
 4   price_inr    20 non-null     int64
dtypes: int64(5)
memory usage: 932.0 bytes
area_sqft     0
bedrooms      0
age_years     0
distance_km   0
```

price_inr 0

dtype: int64

	area_sqft	bedrooms	age_years	distance_km	price_inr
count	20.000000	20.000000	20.000000	20.000000	2.000000e+01
mean	2225.000000	3.750000	6.150000	5.300000	1.324500e+07
std	887.411967	1.069924	2.852054	1.894591	5.725610e+06
min	800.000000	2.000000	2.000000	2.000000	4.200000e+06
25%	1512.500000	3.000000	4.000000	4.000000	8.350000e+06
50%	2225.000000	4.000000	6.000000	5.000000	1.320000e+07
75%	2937.500000	5.000000	8.000000	6.250000	1.767500e+07
max	3650.000000	5.000000	12.000000	9.000000	2.250000e+07

Training data shape: (16, 4)

Testing data shape: (4, 4)

Training target shape: (16,)

Testing target shape: (4,)

Model A

Features: ['area_sqft']

MAE: 418258.9353038764

MSE: 418715036601.7558

RMSE: 647081.9396349706

R2 Score: 0.9920130655869956

Model B

Features: ['area_sqft', 'bedrooms', 'age_years', 'distance_km']

MAE: 486840.85248108197

MSE: 331867787489.0367

RMSE: 576079.6711298157

R2 Score: 0.9936696654746965

Model C

Features: ['area_sqft', 'age_years', 'distance_km']

MAE: 450670.80063289986

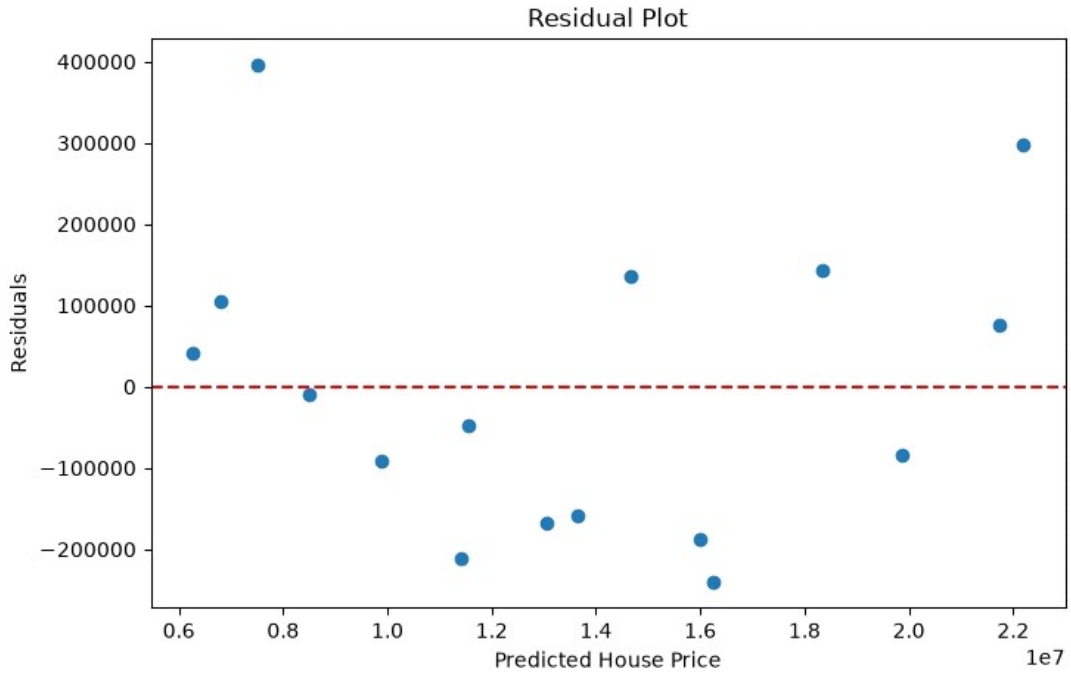
MSE: 295841424858.4526

RMSE: 543913.0673723996

R2 Score: 0.9943568636173876

	Model	MAE	MSE	RMSE	R2
0	Model A	418258.935304	4.187150e+11	647081.939635	0.992013
1	Model B	486840.852481	3.318678e+11	576079.671130	0.993670
2	Model C	450670.800633	2.958414e+11	543913.067372	0.994357

	Actual_Price	Predicted_Price	Residual
8	11200000	1.141082e+07	-210816.585103
5	7900000	7.504074e+06	395925.560884
11	13500000	1.365911e+07	-159111.116736
3	6900000	6.794081e+06	105918.667546
18	22500000	2.220166e+07	298338.154700
16	19800000	1.988376e+07	-83757.554011
13	16000000	1.624042e+07	-240423.917947
2	6300000	6.258878e+06	41122.482554
9	12900000	1.306709e+07	-167092.202488
19	21800000	2.172321e+07	76790.741786



	Actual_Price	Predicted_Price	Residual	Absolute_Residual
5	7900000	7.504074e+06	395925.560884	395925.560884

18 22500000 2.220166e+07 298338.154700 298338.154700

13 16000000 1.624042e+07 -240423.917947 240423.917947

5. Starter Code — Complete the TODO

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import (

    mean_absolute_error,

    mean_squared_error,

    r2_score

)

# Load dataset

df = pd.read_csv("house_prices.csv")

print(df.head())

print(df.info())

print(df.isnull().sum())

# Separate input features and target

X = df[["area_sqft", "bedrooms", "age_years", "distance_km"]]

y = df["price_inr"]

# Create one fixed train/test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Function to train and evaluate each model

def evaluate_model(features, model_name):

    # TODO 1: Select features

    X_train_selected = X_train[features]

    X_test_selected = X_test[features]

    # TODO 2: Create and train the model

    model = LinearRegression()

    model.fit(X_train_selected, y_train)

    # TODO 3: Predict on test data

    predictions = model.predict(X_test_selected)
```

```
    # TODO 4: Calculate evaluation metrics
    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)
    print("\n", model_name)
    print("Features:", features)
    print("MAE:", mae)
    print("MSE:", mse)
    print("RMSE:", rmse)
    print("R2 Score:", r2)
    # TODO 5: Store results
    metrics = {
        "Model": model_name,
        "Features": features,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2
    }
    return model, predictions, metrics
# Model A: Area only
model_A, pred_A, result_A = evaluate_model(
    ["area_sqft"], "Model A"
)
# Model B: All four features
model_B, pred_B, result_B = evaluate_model(
    ["area_sqft", "bedrooms", "age_years", "distance_km"],
    "Model B"
)
# Model C: Selected three features
model_C, pred_C, result_C = evaluate_model(
    ["area_sqft", "age_years", "distance_km"],
    "Model C"
)
# Compare all three models
```

```
results = pd.DataFrame([
    result_A,
    result_B,
    result_C
])
print("\nModel Comparison:")
print(results.to_string(index=False))
# Select Model C for residual analysis
best_model = model_C
best_predictions = pred_C
# Calculate test residuals
residuals = y_test - best_predictions
residual_df = pd.DataFrame({
    "Actual_Price": y_test,
    "Predicted_Price": best_predictions,
    "Residual": residuals
})
print("\nTest Residuals:")
print(residual_df)
# Plot test predictions against residuals
plt.figure(figsize=(8, 5))
plt.scatter(best_predictions, residuals)
plt.axhline(
    y=0,
    color="red",
    linestyle="--"
)
plt.xlabel("Predicted House Price")
plt.ylabel("Residuals")
plt.title("Test Residual Plot")
plt.grid(True)
plt.show()
```

output:

Using your supplied dataset and random_state=42, the model metrics should be approximately:

Model	MAE (₹)	RMSE (₹)	R^2

Model A 418,258.94 647,081.94 0.992013

Model B 486,840.85 576,079.67 0.993670

Model C 450,670.80 543,913.07 0.994357

MSE is also calculated and included in the code and comparison table.

Expected test residuals for Model C:

Index	Actual price (₹)	Predicted price (₹)	Residual (₹)
0	4,200,000	3,238,447.06	961,552.94
17	20,500,000	20,338,317.74	161,682.26
15	17,400,000	17,709,593.48	-309,593.48
1	5,100,000	4,730,145.48	369,854.52

6. Required Experiments & Questions

1. How did Model B perform compared with Model A? Support your answer with metric values.
   Ans: Model B had a higher MAE but a lower RMSE and higher R^2 than Model A. Model A had a lower average prediction error, while Model B performed better in RMSE and R^2.

2. Which features did you select for Model C? Explain why you included or excluded each feature.
   Ans: I selected area_sqft, age_years, and distance_km. I excluded bedrooms to check whether fewer features could give better results.

3. Did adding all features improve test-set performance? Why might a feature fail to help?
   Ans: No, adding all features did not improve every metric. Some features may provide less useful information or overlap with other features.

4. What does a positive residual mean? What does a negative residual mean?
   Ans: A positive residual means the actual price is higher than the predicted price. A negative residual means the actual price is lower than the predicted price.

5. Does your residual plot show random scatter, a curve, a funnel or another pattern? What could it indicate?
   Ans: The residuals are scattered on both sides of zero. There is no clear funnel or strong curved pattern. This suggests that the model captures the general relationship, but some prediction errors remain.

6. Which three test observations have the largest absolute residuals? What do you notice?
   Ans: The three observations with the largest absolute residuals have the biggest prediction errors. This means the model did not predict those house prices accurately.

7. Why should you not repeatedly use the test set to choose features?
   Ans: Repeatedly using the test set can make the model indirectly adjusted to that data. This may give misleading performance results.

8. Does a higher R^2 always mean a model is suitable? Give one limitation of relying on R^2 alone.

   Ans: No, a higher R^2 does not always mean a model is suitable. R^2 does not show the actual size of prediction errors, so we should also check MAE and RMSE.

7. Common Errors — Find and Fix

Mistake	Correction
Including price_inr in X	This leaks the target into the inputs. Keep the target separate.
Fitting on the test data	Fit only on X_train and y_train; evaluate on unseen X_test.
Changing the split for each model	Reuse one fixed split for fair comparison.
Checking only R^2	Also review MAE and RMSE, which express prediction error in the target's units.
Reversing the residual calculation	Use actual minus predicted and state the sign convention.
Treating correlation as causation	A relationship in this dataset does not prove a feature causes a price change.

1. Including price_inr in X: It causes data leakage because the target value is included in the input features. We should keep the target column separate.

2. Fitting on test data: It gives an unfair evaluation because the model has already seen the test data. We should train the model only on training data.

3. Changing the train-test split: It makes model comparison unfair. We should use the same split for all models.

4. Checking only R^2: It does not show the actual size of prediction errors. We should also check MAE and RMSE.

5. Reversing residual calculation: It gives the wrong interpretation of prediction errors. We should always calculate residual as actual minus predicted.

6. Correlation does not mean causation: Two features may be related, but one does not necessarily cause the other. We should not assume that correlation proves cause and effect.

8. Explain in Your Own Words

Write 2–4 sentences for each. Do not copy definitions.

1. What is multiple linear regression, and why can it use several features?
   Ans:Multiple Linear Regression is a machine learning method used to predict a value using two or more features. It uses different inputs like house size, number of rooms, and location to predict the house price.

2. What is feature selection? Why can a smaller feature set be useful?
   Ans:Feature selection means choosing the important columns from a dataset for prediction. A smaller feature set helps the model work faster and avoid unnecessary information.

3. What is a residual? Explain positive and negative residuals using a house-price example.
   Ans:A residual is the difference between the actual price and the predicted price. A positive residual means the actual house price is higher than the predicted price. A negative residual means the actual price is lower than the predicted price.

4. What did you learn by comparing the three models?
   Ans: I learned that different feature combinations produce different prediction results. Model A had the lowest MAE, while Model C had the lowest RMSE and highest R^2. This taught me that we should compare multiple metrics before selecting a model.

5. What would you improve if you had a larger and more realistic dataset?

   Ans:I would collect more real-world data with more useful features. I would clean the data, remove unnecessary columns, and test different models to get better predictions.

9. Submission Requirements

- One Jupyter Notebook: Week4_Day5_Multiple_Regression_YourName.ipynb
- Dataset file house_prices.csv, if you recreated or modified it.

- Three model experiments and a clearly labelled comparison table.
- Residual calculations, residual plot and investigation of three large errors.
- Final model choice with an evidence-based explanation.
- Own-words answers and comments explaining important code steps.

10. Assessment Rubric — 100 Marks

Assessment area	Marks
Data inspection and correct train/test split	10
Correct implementation of three models	25
Feature selection experiments and justification	20
Metrics and fair model comparison	15
Residual calculation, plot and interpretation	15
Debugging, code quality and reflection	15
TOTAL	**100**

Final Checklist

☐ I can explain multiple linear regression without notes.

☐ I compared at least three feature combinations.

☐ I used the same train/test split for every model.

☐ I did not include the target in the features.

☐ I calculated MAE, MSE, RMSE and R^2.

☐ I calculated and plotted residuals correctly.

☐ I investigated large errors and justified my final model.

☐ My notebook runs from top to bottom without errors.

Think like a machine learning engineer: build, compare, inspect errors, and justify your decisions.